

AWS Microservices and CI/CD Pipeline Builder

Project Overview

This repository contains my implementation of the AWS Academy Microservices and CI/CD Pipeline Builder lab.

The lab demonstrates how a monolithic Coffee Suppliers application can be separated into independent Customer and Employee microservices and deployed using AWS cloud services.

Architecture

The implementation uses:

- AWS Cloud9
- AWS CodeCommit
- Docker
- Amazon ECR
- Amazon ECS
- AWS Fargate
- Application Load Balancer
- AWS CodeDeploy
- AWS CodePipeline
- Amazon RDS MySQL

Microservices

Customer Microservice

The Customer Microservice provides customer-facing functionality for viewing supplier information.

Employee Microservice

The Employee Microservice provides administrative functionality for managing supplier information.

It supports operations such as:

- Viewing suppliers
- Adding suppliers
- Editing suppliers
- Managing supplier information

Docker

Separate Docker images were created for:

- customer
- employee

The images were stored in Amazon ECR.

ECS Deployment

The microservices were deployed using Amazon ECS with AWS Fargate.

ECS Cluster:

`microservices-serverlesscluster`

Services:

- `customer-microservice`
- `employee-microservice`

Application Load Balancer

The Application Load Balancer was configured to route traffic between the Customer and Employee microservices.

HTTP : 80

- Default → customer-tg-one
- `/admin/\*` → employee-tg-one

HTTP : 8080

- Default → customer-tg-two
- `/admin/\*` → employee-tg-two

CI/CD

Two CI/CD pipelines were implemented:

- `update-customer-microservice`
- `update-employee-microservice`

AWS CodePipeline and AWS CodeDeploy were used to automate Blue/Green deployments.

Database

The Employee Microservice was connected to an Amazon RDS MySQL database for supplier information.

Source IP Restriction

Access to the Employee `/admin/\*` routes was restricted using a source-IP condition in the Application Load Balancer.

Scaling

The Customer Microservice was scaled to three running tasks.

Final result:

- Desired tasks: 3
- Running tasks: 3
- Healthy tasks: 3
- Unhealthy tasks: 0

Final Result

The project successfully demonstrated:

1. Microservice separation
2. Docker containerization
3. Amazon ECR image management
4. ECS Fargate deployment
5. Application Load Balancer routing
6. Blue/Green deployment
7. CI/CD automation
8. RDS database integration
9. Source-IP access control
10. ECS service scaling

Lab

AWS Academy: Microservices and CI/CD Pipeline Builder